HarnessSpec

Mid-submission progress report

Syed Affan Daimi · M.S. Computer Science, IIIT Hyderabad
CS6.403 Automated Software Engineering · Monsoon 2026
5 November 2026 · registered proposal 21 October 2026
Repository: https://github.com/GauravSarkarr/HarnessSpec

1. What this submission claims

The registered study has three objectives. O1 defines a typed harness. O2 builds the validator, the runtime, and trace-driven revision. O3 compares four designers on about 60 SWE-bench Verified issues and about 60 SWE-QA questions, with 7B–30B local models, three seeds, and energy measured beside accuracy. O3 is most of the experimental work. It is not this submission.

The midpoint freezes the artifact O3 will later measure, and it stops before any accuracy claim. Hypotheses H1–H4 are not estimated. A fixture pilot checks that the compiler and the runtime implement the decisions in the proposal. It is not a benchmark result, and the report does not treat it as one.

Piece	Objective	State at mid-submission
Protocol, eight fields, version 0.1	O1	Frozen. Schema and validator agree.
Validator and bounded repair	O2	Done. Measured on 15 specs.
Budgets, tool guards, hooks, return gate	O2	Done on one scripted workspace.
Trace revision and a spec cache	O2	Mechanism only. No break-even number.
Fixed harness A, and the shape of tuple C	O3 prep	A runs. C is a partial object, not a model.
Free-text briefs (B) and a live orchestrator	O3	Not started.
SWE-bench, SWE-QA, seeds, energy	O3	Not started.

2. Protocol

A HarnessSpec is one JSON object. The eight fields from the proposal are below. `spec_version` and `task_id` are an envelope, not a ninth design choice. The executable check is the validator module. The JSON schema beside it is the interchange file. A unit test fails if their required top-level names diverge.

Field	What the compiler requires
goal	A non-empty statement, and at least one acceptance test whose command is on the runtime allowlist.
context	Include-paths inside the parent slice. Absolute paths and `..` are rejected.
tools	The name exists on the parent card. Permission rank is read, then write, then exec. Paths stay inside that tool's grant. A grant of `src` does not cover `src2`.
budgets	Steps, tokens, and seconds are at least 1 and at most the parent. The token budget covers one token per step.
stop	Known stop conditions. On exhaustion the harness either stops or escalates.
verification	Hooks of kind test, lint, or ast. The command must match the kind.
return_contract	If citations are required, the contract demands at least one.
model	Tier is 7–14B or 24–30B, and that tier is on the parent card.

The runtime will execute exactly three commands: `unittest`, `ast.parse`, and `lint.bare-except`. Anything else is a validation error. There is no shell. A least-privilege allowlist is only real if the runtime knows the program it is about to run. The benchmark phase will add container-pinned commands for SWE-bench. It will not add a free-form shell string.

`examples/valid_spec.json` is one harness that passes the validator against `examples/parent.json`. It is harness D for the adder used in the pilot.

Field	Value in the example
goal	Make add return the sum of its arguments. One acceptance test, command unittest.
context	Include src and tests. Exclude nothing.
tools	read_file at read, on src and tests. write_file at write, on src only. No exec tool.
budgets	6 steps, 2000 tokens, 30 seconds. All inside the parent card.
stop	Stop on acceptance, budget, hook failure, or a rejected return. On exhaustion, escalate. At most one escalation.
verification	unittest, then ast.parse.
return_contract	Citations required, test output required, at least one citation.
model	Tier 7–14B, name fixture-small. No digest yet. Digests belong to the pinned-model runs.

Harness A, the fixed generic designer, is the same envelope with every parent tool, the parent's full budget, no hooks, and citations not required. A unit test builds A and D and requires both to validate. The example file is that D harness, written out. A free-text brief is stored and marked not compiled. There is no parser for it.

3. Compiler measurements

Every figure in this section is from `pilot/results.json`, produced by `pilot/run_pilot.py` on Python 3.12.10. The parent card is the one in `examples/parent.json`: `read_file` (read on `src`, `tests`, `README.md`), `write_file` (write on `src`), `list_dir` (read on the workspace), `run_tests` (exec on `tests`), 30 steps, 8000 tokens, 120 seconds, both open-weight tiers.

Fifteen hand-written specs were validated, then passed through repair of at most three rounds. Four were valid on the first pass. Seven more were valid after one round. Four were still invalid when repair stopped. The cap is three rounds. None of these cases used the third round: two invalid specs changed nothing on round 1 and stopped, and two changed once, then stopped on round 2 because the spec no longer moved.

Spec	Initial	Repaired	What happened
Four well-formed harnesses	valid	unchanged	Left untouched.
Unknown tool, exec on a write tool, budget above the parent, tokens below the step count, tier 70B, extra field, write path outside `src`	invalid	valid, 1 round	Drop, downgrade, clamp, or delete. Privileges only move down.
Empty acceptance list	invalid	still invalid	`goal.acceptance_tests` remains. Tests are not invented.
Blank goal statement	invalid	still invalid	`goal.empty` remains. The statement is not invented.
Include path `../secrets`	invalid	stops at round 2	The path is deleted. The slice is then empty. A slice is not invented.
AOrchestra-style tuple: instruction, context, tool names, model name	invalid	stops at round 2	Budgets, stop, verification, return contract, and tier are filled with conservative defaults. Repair stops at `schema.missing:goal.acceptance_tests`.

The tuple row is the gap the proposal named against AOrchestra, shown as a property of this compiler. It is not a claim about any model, and it is not an accuracy result. Repair fills the fields that have a safe default, including a strict return contract (citations and test output required) and escalation on exhaustion. It refuses to supply the test the tuple never contained.

Determinism: the unknown-tool spec was validated 100 times. There was one unique verdict, `tool.unknown`. The validator does not read a clock or call a model.

This changes how O3 has to be run. Condition C must be scored as the tuple the orchestrator emitted. If the experiment repaired C until it became D, the comparison would collapse. The repair study above is a compiler result. It is not experimental condition C.

4. Runtime measurements

The workspace is `fixtures/off_by_one`. `src/app.py` returns `a + b + 1`. `tests/test_app.py` expects `a + b`. The same scripted actions were submitted to two harnesses. No model chose them. The question is what the runtime accepts, not whether an agent can solve the task.

Harness A is the fixed generic harness: every tool on the parent card, the parent's full budget, no verification hooks, citations not required. Harness D is a full spec: read and write only, a tight step budget where the budget case needs one, `unittest` and `ast.parse` hooks, and a return contract of at least one citation plus test output. Both validate before the runtime will start. An invalid spec is refused.

Scripted task	Harness A	Harness D
Patch the file, then return “the bug is fixed” with no citation	Accepted. Counted as a silent handoff. Hooks: 0.	Both hooks passed (the patch was real). The return was then rejected. Not a silent handoff.
Three writes, D capped at 2 steps	All 3 writes applied. No budget violation.	Third write intercepted. 1 budget violation. 2 steps used. Stop action: escalate.
`run_tests` as exec	Applied. The parent card allows it, and A grants it.	1 tool denial. D's allowlist has no exec tool.
Return a cited claim while the test still fails	Accepted. Tests were not run.	Not accepted. `unittest` failed. Stop: hook_failed.
Patch the file, then return `src/app.py:2` and test output	Not the contrast of interest. A does not consult the test.	Both hooks passed. Return accepted. Not a silent handoff.

Two distinctions matter, and they are easy to blur. On the first row the patch landed under both harnesses, so the test was able to pass. A still accepted a confident summary with no evidence. D refused that summary after the hooks had passed. On the fourth row the workspace was still wrong. A accepted a cited claim without running the test. D ran it and refused the return. On the fifth row D accepted a return only when the test passed and the contract's evidence was present. That is the gate working in both directions.

These are nine scripted runs, one workspace, one seed, and zero model tokens. They do not support H1 or H2. They define the instruments those hypotheses need: a silent-handoff bit, a budget-violation count, a tool-denial count, and a hook failure. H2's comparison is against B and C, which were not run.

Revision was exercised on the two-step harness. The third action was a budget violation. `revise_from_trace` raised `max_steps` from 2 to 4, inside the parent cap of 30, and left `require_citations` true with `min_citations` 1. The revised spec still validates. This revision will not raise a tool privilege and will not drop the citation rule. A failed return stays visible to the orchestrator.

The spec cache recorded one miss and one hit for task type `bugfix` at tier 7–14B. The hit returns the stored spec and does not re-enter repair. Hypothesis H4, the break-even count of tasks that amortise a design call, is not estimated. There is no measured design-call cost. The repairer in this snapshot is deterministic and free. Reporting that as a token saving would be false. H1's token comparison has the same gap: the design call is defined to count, and no design call was made.

5. The four hypotheses, unchanged

The proposal pre-registered four hypotheses and said a negative result would be reported as one. This submission does not revise them. It records which instrument now exists, and which quantity is still missing. None of the four is a finding.

Id	Registered claim	Instrument that now exists	Not available yet
H1	D stays within 5 points of A on accuracy and uses fewer tokens, design call included.	A and D can both be executed on a scripted task.	Accuracy. Model tokens. A design call.
H2	D has fewer silent handoffs than B and C.	A silent-handoff bit. On the fixture, A accepts an unsupported summary and D rejects it.	B, C, and any model-written return.
H3	Validation plus revision raises first-pass validity, most for a 7–14B orchestrator.	Typed errors, a three-round cap, and a validity count on 15 specs.	An orchestrator. The 15 specs were written by hand.
H4	Report how many tasks a cached spec must cover before the design call is paid for.	A cache, with one miss and one hit.	A non-zero design cost. Break-even is undefined, not zero.

A negative result, if it comes, will be stated in those terms. If D falls more than 5 points behind A, H1 fails. If silent handoffs are not lower for D than for B and C, H2 fails. If first-pass validity does not rise, H3 fails, including the case where a small model rarely emits an acceptance test. If the design call is never paid back inside the task set, H4 fails and the report will say the break-even was not reached. Repair will not be used to convert any of those outcomes into a pass.

6. Deviations from the registered plan

Six choices are narrower than the proposal, and each one is there so the midpoint cannot pretend to be the study.

Validator. The check is Python, not a JSON Schema library. The schema file is the published contract. A test locks the required names. Nested rules live in the validator, because that is the program the runtime actually trusts.

Clock. Time budgets are read from an injected clock. The pilot uses a fake clock, so a pass is not a race against the machine. A wall clock is in the runtime for a later live run. It was not used for any number above.

Hooks. The proposal names tests, linters, and AST checks. All three kinds exist. The commands are an allowlist of three. A spec cannot carry a shell string.

Revision policy. The proposal left the edit rule open, beyond “trace-driven.” This code may raise a step budget up to the parent cap. It will not grant a broader tool, and it will not drop a citation requirement. A trace cannot talk the compiler out of least privilege.

Condition B. A free-text brief is stored and flagged as not compiled. A parser would be a hidden designer, so there is not one.

Condition C. Repair of a four-field tuple fills budgets and a strict return contract, then stops on the missing test. That repaired object is not condition C. Scoring it as C would erase the contrast. The run condition, later, is the tuple as emitted.

7. What the repository is for

The history on github.com/GauravSarkarr/HarnessSpec is the construction of this snapshot, on 5 November 2026, after the proposal of 21 October. It is not a backdated calendar.

Commit	What it adds
`5ec1ef5`	Schema 0.1, path rules, parent capability card, the registered proposal PDF.
`e3ea1db`	Validator. Unknown tools, excess privilege, and unaffordable budgets become typed errors.
`47c8a7a`	Repair, capped at three rounds, with no invention of a goal or a test.
`aca97ab`	Runtime, hooks, and the return gate.
`02c1967`	Harness A, the four-field tuple, and conservative trace revision.
`a60e33e`	Fixture pilot. `pilot/results.json` is locked by a test that re-runs it.
`aa8204d`	README and `docs/AI_USE.md`.

Reproduce the numbers, from a clean checkout, with no installed packages:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
PYTHONPATH=src python3 pilot/run_pilot.py
PYTHONPATH=src python3 -m harnessspec examples/valid_spec.json examples/parent.json
```

The test suite re-runs the pilot and fails if `pilot/results.json` has drifted.

Every number in this report was taken from that file after a passing run on Python 3.12.10.

AI-tool use, which the proposal said would be logged: Cursor, Grok 4.7, on 5 November 2026, drafted the milestone implementation and this report. The check is the command above. No orchestrator model and no subagent model were called. The log is `docs/AI_USE.md`.

8. Remaining work, in the order it should happen

The second half is the study that was registered, not a larger version of the fixture.

1. A local orchestrator, measured on validity. One 7–14B model must emit a HarnessSpec. Typed errors go back to it, at most three rounds. The quantities are first-pass validity and rounds used. That is the setup for H3. A small n will be reported as a pilot, not as H3.

2. B and C as run conditions. B stays a free-text brief and is not compiled. C stays the four-field tuple, unrevised, so a missing acceptance test is a failed design. D is the full spec plus the validator. Scoring repaired-C as if it were C would erase the contrast in section 3.

3. Ten SWE-bench Verified instances through the real edit-and-test loop, before the registered sizes of about 60 and about 60. The point of the ten is to learn which commands the allowlist must grow to. SWE-QA comes after that slice holds.

4. Accounting the proposal already fixed. Tokens from the server, design call included. Three seeds. Paired bootstrap intervals. CPU and GPU energy beside every accuracy figure. The cache break-even (H4) waits until a design call has a real token cost.

5. Hooks. The three-command allowlist grows to the pinned commands those benchmarks need. It does not become a general shell.

9. Risks, with what the midpoint changed

The risks in the proposal still stand. A small orchestrator may write poor specs, and the design call may cost more than the harness saves. Two of them are sharper now.

The compiler will not hide a missing test. If a 7–14B orchestrator rarely emits an acceptance test, the first-pass validity of D stays low. That is a reportable result. Patching it inside repair would falsify H3.

The fixture hook is `unittest discover` inside one workspace. SWE-bench tests are not that command. The runtime can be right on `fixtures/off_by_one` and still fail to run the benchmark. That is the main integration risk of the second half, and it is why the next milestone is ten instances rather than the full set.

Nothing here estimates success per kilowatt-hour, or tokens per correct answer. Those columns stay empty until the runs exist.

10. Where the project stands

O1 is met: HarnessSpec 0.1 is a typed eight-field protocol with a deterministic compiler. The enforceable half of O2 is met on fixtures: budgets, tool grants, hooks, and the return contract all change what a scripted subagent is allowed to do, and repair will not invent the evidence the protocol exists to demand. O3 has not started. H1–H4 are not results of this submission. The repository is the protocol, the compiler, the runtime, and the fixture evidence above.